LAPORAN
WORKSHOP SISTEM INFORMASI WEB SERVER
(TIF320803)
MINGGU 8 ACARA 17

SEMESTER 3



Tugas :
API CRUD

Disusun Oleh:
Uaryl Engno Laurenca (E41250317)

PROGRAM STUDI DILUAR KAMPUS UTAMA (PSDKU)
TEKNIK INFORMATIKA KAB. NGANJUK
JURUSAN TEKNOLOGI INFORMASI
POLITEKNIK NEGERI JEMBER
TAHUN 2026

A. PENDAHULUAN

Aplikasi web modern jarang berdiri sendiri. Data yang sama sering dipakai oleh banyak klien sekaligus, seperti halaman web, aplikasi mobile, atau layanan lain. Untuk itu server menyediakan API (Application Programming Interface) sebagai media komunikasi yang terstandarisasi, dengan HTTP sebagai protokol dan JSON sebagai format pertukaran data.

Pada Acara 17 ini dibangun API CRUD (Create, Read, Update, Delete) dengan PHP Native, yaitu PHP murni tanpa framework. Pendekatan ini dipilih agar alur kerja API terlihat jelas dari awal sampai akhir: menerima request, memproses logika, mengakses database, lalu mengirim response. Kode disusun dengan pola MVC (Model-View-Controller) dan Front Controller. Struktur ini menjadi fondasi untuk acara-acara berikutnya.

Resource utama yang dikelola adalah data mahasiswa. Sebagai tugas mandiri, ditambahkan resource kedua yaitu buku, validasi NIM agar tidak duplikat, dan dokumentasi endpoint.

B. TUJUAN PRAKTIKUM

1. Memahami konsep API, CRUD, dan pemetaan operasi CRUD ke HTTP method.
2. Membangun struktur proyek berpola MVC dengan folder app/ terpisah dari public/.
3. Membuat lapisan Model (koneksi PDO dan prepared statement) dan lapisan Controller (logika endpoint).
4. Membuat Autoloader (spl_autoload_register), Router, dan Front Controller sebagai satu-satunya titik masuk aplikasi.
5. Menguji endpoint CRUD memakai Postman.
6. Mengembangkan resource baru (Buku) dan validasi NIM duplikat sesuai pola yang sama.

C. DASAR TEORI

API dan PHP Native

API adalah kumpulan aturan dan mekanisme yang memungkinkan dua aplikasi bertukar data tanpa mengetahui implementasi internal masing-masing. PHP Native berarti kode PHP ditulis tanpa framework seperti Laravel atau CodeIgniter, sehingga mahasiswa memahami langsung proses dari menerima request sampai mengirim response.

API CRUD

CRUD adalah empat operasi dasar pada data. Pemetaannya ke HTTP method:

- Create : POST
- Read : GET
- Update : PUT/PATCH
- Delete : DELETE
    - **Arsitektur MVC**
- Model: mengurus data dan interaksi dengan database (query).
- View: mengurus tampilan. Pada acara ini belum dipakai karena response berupa JSON.

- Controller: menerima request, memanggil Model, lalu menentukan response. Controller tidak boleh menulis SQL langsung.

Autoloader
spl_autoload_register() mendaftarkan fungsi yang dipanggil otomatis oleh PHP saat sebuah class dipakai tetapi belum dimuat. Dengan begitu class cukup dipanggil dengan new NamaClass() tanpa require manual.

Front Controller dan Router
Front Controller adalah satu titik masuk tunggal (public/index.php) yang menerima seluruh request. Router kemudian membaca URL dan HTTP method, lalu meneruskannya ke Controller yang sesuai. File .htaccess mengarahkan semua request yang bukan file atau folder nyata ke index.php, sehingga URL cukup /mahasiswa tanpa ekstensi .php.

JSON
JSON adalah format pertukaran data yang ringan dan tidak bergantung pada bahasa pemrograman. Di PHP, data dikonversi ke JSON dengan json_encode(), sedangkan body JSON dari client dibaca dengan file_get_contents('php://input') lalu json_decode().

PDO dan Prepared Statement
PDO (PHP Data Objects) adalah antarmuka seragam untuk mengakses berbagai database. Prepared statement memisahkan struktur SQL dari data yang dimasukkan pengguna melalui placeholder (:nama atau ?), sehingga mencegah SQL Injection.

Alur Kerja API CRUD
Client mengirim HTTP Request, lalu PHP Native API memprosesnya, mengakses database MySQL, mengubah hasilnya menjadi JSON, dan mengirim JSON Response ke client.

D. HASIL DAN PEMBAHASAN

Alat dan Bahan
XAMPP (Apache, PHP, MySQL), Visual Studio Code, Git dan GitHub, web browser, dan Postman.

Struktur Folder Proyek
Proyek dibuat di htdocs/sistem-informasi-akademik dengan struktur berikut:
app/config/Database.php : koneksi PDO (Singleton)
app/core/autoload.php : autoloader class
app/core/Router.php : memetakan URL ke Controller
app/core/BaseController.php : helper bersama (json, readJson, requireId)
app/models/Mahasiswa.php : query tabel mahasiswa
app/models/Buku.php : query tabel buku (tugas mandiri)
app/controllers/MahasiswaController.php : logika endpoint mahasiswa
app/controllers/BukuController.php : logika endpoint buku (tugas mandiri)
public/index.php : Front Controller
public/.htaccess : mengarahkan semua request ke index.php

views/ : disiapkan untuk acara berikutnya
db_workshop.sql : skrip database
Folder app/ berada di luar public/. Ini praktik keamanan penting: jika saat deployment document root hanya diarahkan ke public/, maka seluruh kode sumber, termasuk kredensial database, tidak bisa diakses langsung lewat browser.
[SCREENSHOT 3: struktur folder proyek di VS Code (Explorer)]

Database
Database db_workshop dibuat dengan skrip db_workshop.sql. Tabel mahasiswa berisi kolom id (primary key, auto increment), nim, nama, prodi, dan created_at. Kolom nim diberi UNIQUE KEY sebagai pengaman tambahan untuk tugas validasi NIM. Tabel buku berisi id, judul, penulis, tahun, dan created_at.

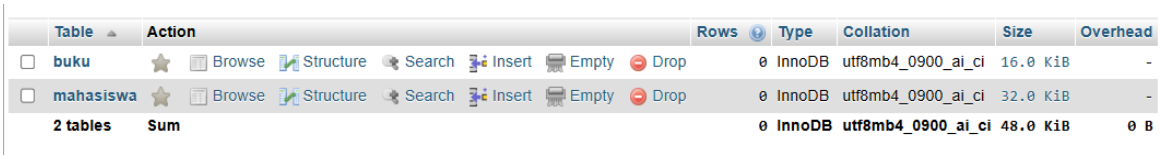
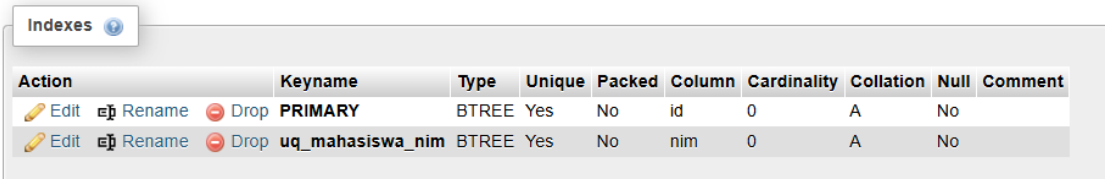


Table	Action	Rows	Type	Collation	Size	Overhead
buku	Browse Structure Search Insert Empty Drop	0	InnoDB	utf8mb4_0900_ai_ci	16.0 KiB	-
mahasiswa	Browse Structure Search Insert Empty Drop	0	InnoDB	utf8mb4_0900_ai_ci	32.0 KiB	-
2 tables	Sum	0	InnoDB	utf8mb4_0900_ai_ci	48.0 KiB	0 B

[SCREENSHOT 4: database db_workshop dan tabel mahasiswa serta buku di phpMyAdmin]



Indexes

Action	Keyname	Type	Unique	Packed	Column	Cardinality	Collation	Null	Comment
Edit Rename Drop	PRIMARY	BTREE	Yes	No	id	0	A	No	
Edit Rename Drop	uq_mahasiswa_nim	BTREE	Yes	No	nim	0	A	No	

[SCREENSHOT 5: struktur tabel mahasiswa (tab Structure), terlihat index unik pada nim]

Implementasi Setiap Lapisan
a. Model: koneksi database (Database.php). Koneksi dibuat sebagai class dengan pola Singleton sederhana sehingga koneksi tidak dibuka berulang. Mode error PDO diatur ke exception, dan jika koneksi gagal, client hanya menerima pesan umum “Koneksi database gagal” dengan status 500.

ACARA17 > app > config > Database.php > ...

```php
<?php
class Database {
    private static ?PDO $connection = null;

    public static function getConnection(): PDO {
        if (self::$connection === null) {
            $host = 'localhost';
            $db   = 'db_workshop';
            $charset = 'utf8mb4';
            $dsn = "mysql:host=$host;dbname=$db;charset=$charset";
            $options = [
                PDO::ATTR_ERRMODE            => PDO::ERRMODE_EXCEPTION,
                PDO::ATTR_DEFAULT_FETCH_MODE => PDO::FETCH_ASSOC,
            ];
            try {
                // Sesuaikan user/password jika MySQL kamu memakai password.
                self::$connection = new PDO($dsn, 'root', '', $options);
            } catch (PDOException $e) {
                http_response_code(500);
                header('Content-Type: application/json');
                echo json_encode(['success' => false, 'message' => 'Koneksi database gagal']);
                exit;
            }
        }
        return self::$connection;
    }
}

```

[SCREENSHOT 6: kode Database.php]

b. Model: Mahasiswa.php. Seluruh perintah SQL hanya berada di Model dan memakai prepared statement. Method yang dibuat: getAll(), find(), create(), update(), delete(), dan nimExists() (untuk tugas mandiri).

ACARA17 > app > models > Mahasiswa.php > ...

```php
<?php
class Mahasiswa {
    private PDO $db;

    public function __construct() {
        $this->db = Database::getConnection();
    }

    public function getAll(): array {
        return $this->db->query('SELECT * FROM mahasiswa ORDER BY id DESC')->fetchAll();
    }

    public function find(int $id) {
        $stmt = $this->db->prepare('SELECT * FROM mahasiswa WHERE id = :id');
        $stmt->execute([':id' => $id]);
        return $stmt->fetch();
    }

    /**
     * Tugas mandiri no. 2: cek apakah NIM sudah dipakai.
     * $exceptId dipakai saat UPDATE agar NIM milik data itu sendiri tidak dianggap duplikat.
     */
    public function nimExists(string $nim, ?int $exceptId = null): bool {
        if ($exceptId === null) {
            $stmt = $this->db->prepare('SELECT COUNT(*) FROM mahasiswa WHERE nim = :nim');
            $stmt->execute([':nim' => $nim]);
        } else {
            $stmt = $this->db->prepare('SELECT COUNT(*) FROM mahasiswa WHERE nim = :nim AND id <> :id');
            $stmt->execute([':nim' => $nim, ':id' => $exceptId]);
        }
        return (int) $stmt->fetchColumn() > 0;
    }

    public function create(array $data): string {
        $stmt = $this->db->prepare(
            'INSERT INTO mahasiswa (nim, nama, prodi) VALUES (:nim, :nama, :prodi)'
        );
        $stmt->execute([':nim' => $data['nim'], ':nama' => $data['nama'], ':prodi' => $data['prodi']]);
        return $this->db->lastInsertId();
    }

    public function update(int $id, array $data): void {
        $stmt = $this->db->prepare(
            'UPDATE mahasiswa SET nim = :nim, nama = :nama, prodi = :prodi WHERE id = :id'
        );
        $stmt->execute([':nim' => $data['nim'], ':nama' => $data['nama'], ':prodi' => $data['prodi'], ':id' => $id]);
    }

    public function delete(int $id): void {
        $stmt = $this->db->prepare('DELETE FROM mahasiswa WHERE id = :id');
        $stmt->execute([':id' => $id]);
    }
}

```

[SCREENSHOT 7: kode Mahasiswa.php]

c. Controller: MahasiswaController.php. Controller tidak menulis SQL. Method index() menampilkan semua data atau satu data bila ada ?id=, store() menambah data, update($id) mengubah data, dan destroy($id) menghapus data. Validasi dilakukan sebelum Model dipanggil, dan status code dikirim sesuai hasilnya.

ACARA17 > app > controllers > MahasiswaController.php > ...

```php
<?php
class MahasiswaController extends BaseController {
    private Mahasiswa $model;

    public function __construct() {
        $this->model = new Mahasiswa();
    }

    public function index(): void {
        $id = $_GET['id'] ?? null;

        if ($id !== null && $id !== '') {
            $data = $this->model->find($this->requireId($id));
            if (!$data) {
                $this->json(404, ['success' => false, 'message' => 'Data tidak ditemukan']);
            }
            $this->json(200, ['success' => true, 'data' => $data]);
        }

        $this->json(200, ['success' => true, 'data' => $this->model->getAll()]);
    }

    public function store(): void {
        $data = $this->clean($this->readJson());

        if ($data['nim'] === '' || $data['nama'] === '' || $data['prodi'] === '') {
            $this->json(400, ['success' => false, 'message' => 'Field nim, nama, dan prodi wajib diisi']);
        }
        if (strlen($data['nim']) > 15) {
            $this->json(400, ['success' => false, 'message' => 'NIM maksimal 15 karakter']);
        }
        // Tugas mandiri no. 2: NIM tidak boleh duplikat
        if ($this->model->nimExists($data['nim'])) {
            $this->json(409, ['success' => false, 'message' => 'NIM sudah terdaftar']);
        }

        $id = $this->model->create($data);
        $this->json(201, ['success' => true, 'message' => 'Data berhasil ditambahkan', 'data' => ['id' => (int) $id]]);
    }

    public function update($id): void {
        $id = $this->requireId($id);
        if (!$this->model->find($id)) {
            $this->json(404, ['success' => false, 'message' => 'Data tidak ditemukan']);
        }

        $data = $this->clean($this->readJson());
        if ($data['nim'] === '' || $data['nama'] === '' || $data['prodi'] === '') {
            $this->json(400, ['success' => false, 'message' => 'Field nim, nama, dan prodi wajib diisi']);
        }
        if (strlen($data['nim']) > 15) {
            $this->json(400, ['success' => false, 'message' => 'NIM maksimal 15 karakter']);
        }
        if ($this->model->nimExists($data['nim'], $id)) {
            $this->json(409, ['success' => false, 'message' => 'NIM sudah dipakai mahasiswa lain']);
        }

        $this->model->update($id, $data);
        $this->json(200, ['success' => true, 'message' => 'Data berhasil diperbarui']);
    }

    public function destroy($id): void {
        $id = $this->requireId($id);
        if (!$this->model->find($id)) {
            $this->json(404, ['success' => false, 'message' => 'Data tidak ditemukan']);
        }

        $this->model->delete($id);
        $this->json(200, ['success' => true, 'message' => 'Data berhasil dihapus']);
    }

    /** Ambil hanya field yang dipakai dan rapikan spasi. */
    private function clean(array $in): array {
        $out = [];
        foreach (['nim', 'nama', 'prodi'] as $f) {
            $out[$f] = isset($in[$f]) && is_scalar($in[$f]) ? trim((string) $in[$f]) : '';
        }
        return $out;
    }
}
```

[SCREENSHOT 8: kode MahasiswaController.php]

d. Autoloader dan Router. Autoloader mencari class di folder config, core, models, dan controllers. Router membaca nama resource dari ujung URL, mencocokkannya dengan peta resource, lalu memanggil method Controller sesuai HTTP method: GET ke index(), POST ke store(), PUT ke update(), dan DELETE ke destroy(). Method lain dibalas 405, dan resource yang tidak dikenal dibalas 404.

ACARA17 > app > core > Router.php > ...

```php
<?php
class Router {
    // Peta sederhana: nama resource pada URL => nama class Controller
    private array $map = [
        'mahasiswa' => 'MahasiswaController',
        'buku'      => 'BukuController', // tugas mandiri no. 1
    ];

    public function dispatch(): void {
        $uri = trim(parse_url($_SERVER['REQUEST_URI'], PHP_URL_PATH), '/');
        $segments = $uri === '' ? [] : explode('/', $uri);

        // Cari nama resource dari ujung URL, sehingga tetap jalan di folder
        // mana pun proyek ditaruh (mis. /sistem-informasi-akademik/public/mahasiswa).
        $controllerName = null;
        $last = end($segments);
        if ($last !== false && isset($this->map[$last])) {
            $controllerName = $this->map[$last];
        }

        if (!$controllerName) {
            $this->json(404, ['success' => false, 'message' => 'Endpoint tidak ditemukan']);
        }

        $controller = new $controllerName();

        switch ($_SERVER['REQUEST_METHOD']) {
            case 'GET':    $controller->index(); break;
            case 'POST':   $controller->store(); break;
            case 'PUT':    $controller->update($_GET['id'] ?? null); break;
            case 'DELETE': $controller->destroy($_GET['id'] ?? null); break;
            default:
                header('Allow: GET, POST, PUT, DELETE');
                $this->json(405, ['success' => false, 'message' => 'Method tidak diizinkan']);
        }
    }

    private function json(int $status, array $body): void {
        http_response_code($status);
        header('Content-Type: application/json; charset=utf-8');
        echo json_encode($body, JSON_UNESCAPED_UNICODE);
        exit;
    }
}

```

ACARA17 > app > core > autoload.php > ...

```php
<?php
spl_autoload_register(function ($class) {
    $folders = ['config', 'core', 'models', 'controllers'];
    foreach ($folders as $folder) {
        $path = __DIR__ . "/../{$folder}/{$class}.php";
        if (file_exists($path)) {
            require $path;
            return;
        }
    }
});

```

[SCREENSHOT 9: kode Router.php dan autoload.php]

e. Front Controller. public/index.php memuat autoloader lalu menjalankan (new Router())->dispatch(). File .htaccess mengarahkan semua request non-file ke file ini.

ACARA17 > public > index.php > ...

```php
<?php
// Front Controller: satu-satunya pintu masuk untuk seluruh request API.
require_once __DIR__ . '/../app/core/autoload.php';

// Jangan bocorkan error PHP mentah ke client (merusak format JSON).
ini_set('display_errors', '0');

try {
    (new Router())->dispatch();
} catch (Throwable $e) {
    // Mis. query gagal karena pelanggaran constraint; detail tidak ditampilkan ke client.
    error_log($e->getMessage());
    http_response_code(500);
    header('Content-Type: application/json; charset=utf-8');
    echo json_encode(['success' => false, 'message' => 'Terjadi kesalahan pada server']);
}

```

ACARA17 > public > .htaccess

```
RewriteEngine On
RewriteCond %{REQUEST_FILENAME} !-f
RewriteCond %{REQUEST_FILENAME} !-d
RewriteRule ^ index.php [QSA,L]

```

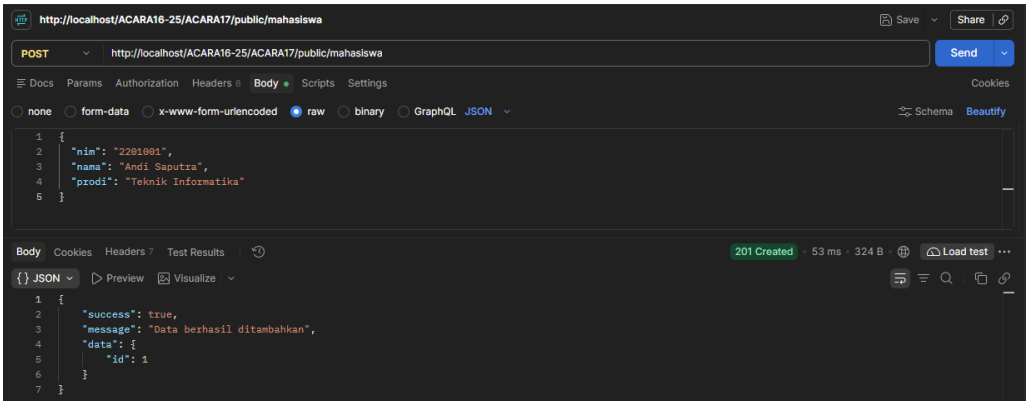
[SCREENSHOT 10: kode public/index.php dan .htaccess]

Diagram Alur Router, Controller, Model

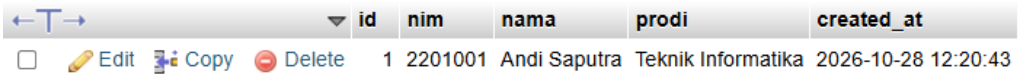
Client (Postman)
→ HTTP Request: METHOD + /mahasiswa[?id=n] + body JSON
→ .htaccess mengarahkan ke public/index.php (Front Controller)
→ Router::dispatch() membaca URL dan HTTP method
→ Controller (index / store / update / destroy), tanpa SQL
→ Model (getAll / find / create / update / delete / nimExists)
→ Database::getConnection() (PDO) → MySQL db_workshop (prepared statement)
→ Controller membentuk Response JSON + status code
→ Client
Contoh alur POST: Router meneruskan ke MahasiswaController::store(), yang memvalidasi data lalu memanggil Mahasiswa::nimExists() dan Mahasiswa::create(), lalu mengirim response 201.

Pengujian dengan Postman

Base URL: http://localhost/ACARA16-25/ACARA17/public/mahasiswa
(1) POST tambah data
POST /mahasiswa, Body, raw, JSON:
{“nim”:”2201001",”nama”:”Andi Saputra”,”prodi”:”Teknik Informatika”}
Hasil: status 201 Created, message “Data berhasil ditambahkan”, dan id data baru.

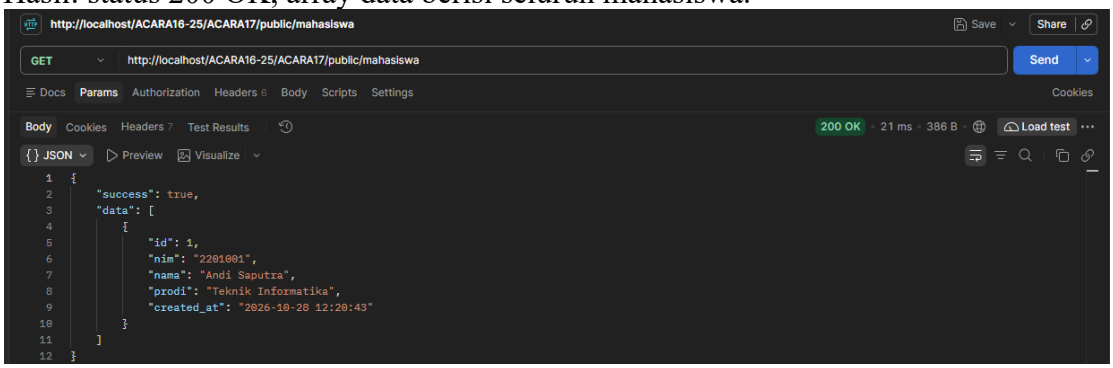


[SCREENSHOT 12: Postman POST, body request dan response 201]



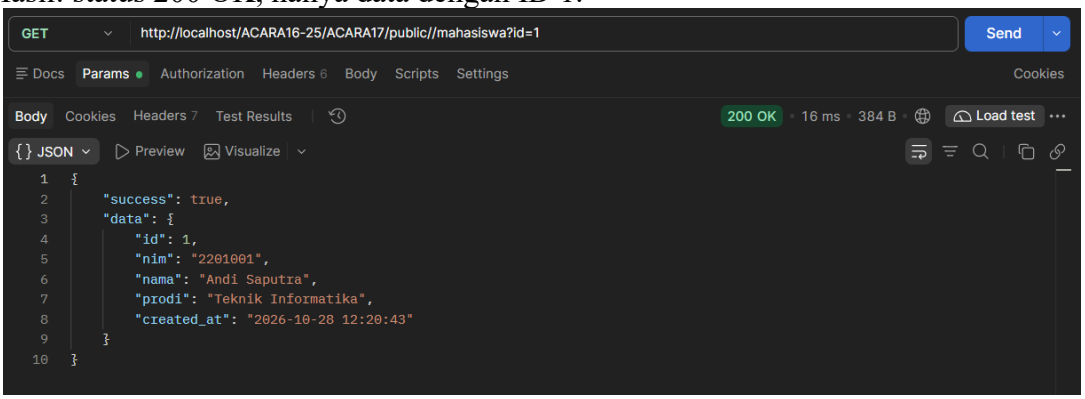
[SCREENSHOT 13: phpMyAdmin menunjukkan data Andi Saputra di tabel mahasiswa]

(2) GET semua data
GET /mahasiswa
Hasil: status 200 OK, array data berisi seluruh mahasiswa.



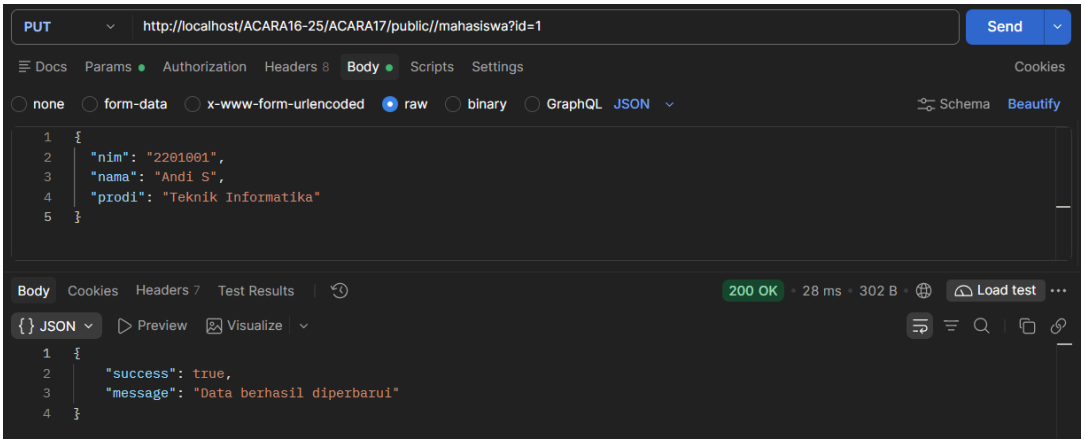
[SCREENSHOT 14: Postman GET /mahasiswa, status 200]

(3) GET satu data
GET /mahasiswa?id=1
Hasil: status 200 OK, hanya data dengan ID 1.

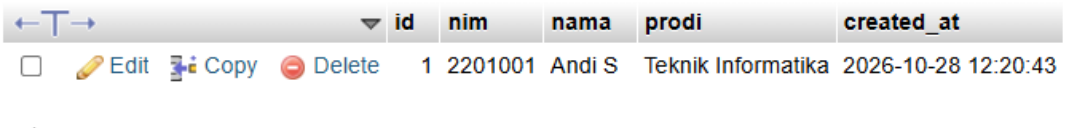


[SCREENSHOT 15: Postman GET /mahasiswa?id=1, status 200]

(4) PUT ubah data
PUT /mahasiswa?id=1, Body:
{“nim”:”2201001",”nama”:”Andi S”,”prodi”:”Teknik Informatika”}
Hasil: status 200 OK, message “Data berhasil diperbarui”.

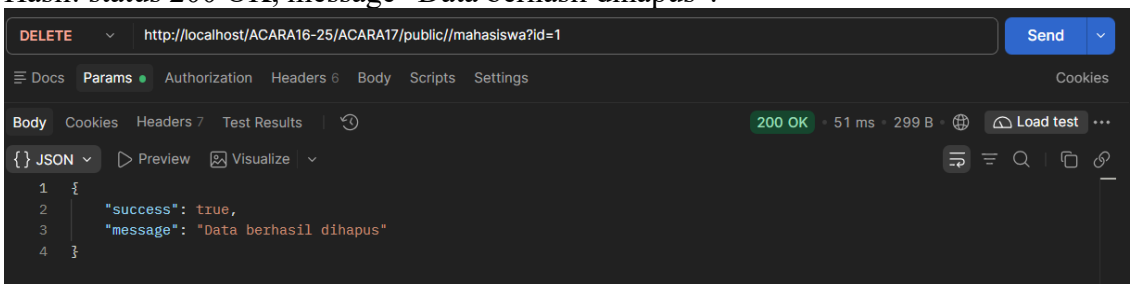


[SCREENSHOT 16: Postman PUT, body request dan response 200]



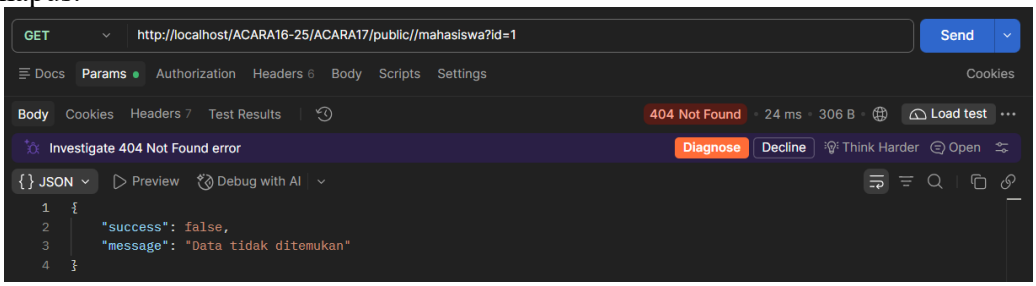
[SCREENSHOT 17: phpMyAdmin menunjukkan nama sudah berubah]

(5) DELETE hapus data
DELETE /mahasiswa?id=1
Hasil: status 200 OK, message “Data berhasil dihapus”.



[SCREENSHOT 18: Postman DELETE, status 200]

(6) GET ulang setelah DELETE
GET /mahasiswa?id=1
Hasil: status 404, message “Data tidak ditemukan”. Ini membuktikan data sudah terhapus.



[SCREENSHOT 19: Postman GET ?id=1 setelah delete, status 404]

Tugas Praktik Mandiri

Tugas 1: Resource Buku. Dibuat Buku.php (Model) dan BukuController.php (Controller) dengan pola yang sama, lalu resource buku didaftarkan di peta Router. Field judul, penulis, dan tahun wajib diisi, dan tahun divalidasi sebagai angka 4 digit. POST /buku dengan {“judul”:”Belajar PHP”,”penulis”:”Budi”,”tahun”:2023} menghasilkan status 201.



ACARA17 > app > controllers > BukuController.php > ...

```php
<?php
// Tugas mandiri no. 1: controller untuk resource 'buku'.
class BukuController extends BaseController {
    private Buku $model;

    public function __construct() {
        $this->model = new Buku();
    }

    public function index(): void {
        $id = $_GET['id'] ?? null;

        if ($id !== null && $id !== '') {
            $data = $this->model->find($this->requireId($id));
            if (!$data) {
                $this->json(404, ['success' => false, 'message' => 'Data tidak ditemukan']);
            }
            $this->json(200, ['success' => true, 'data' => $data]);
        }

        $this->json(200, ['success' => true, 'data' => $this->model->getAll()]);
    }

    public function store(): void {
        $data = $this->validated($this->readJson());
        $id = $this->model->create($data);
        $this->json(201, ['success' => true, 'message' => 'Data berhasil ditambahkan', 'data' => ['id' => (int) $id]]);
    }

    public function update($id): void {
        $id = $this->requireId($id);
        if (!$this->model->find($id)) {
            $this->json(404, ['success' => false, 'message' => 'Data tidak ditemukan']);
        }

        $this->model->update($id, $this->validated($this->readJson()));
        $this->json(200, ['success' => true, 'message' => 'Data berhasil diperbarui']);
    }

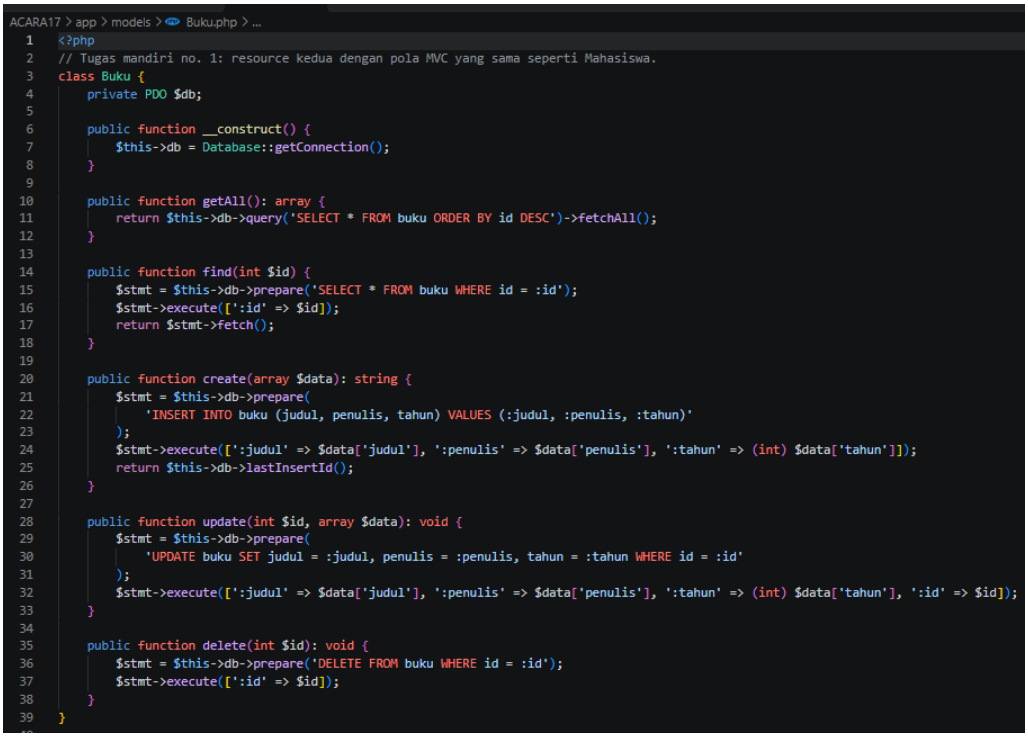
    public function destroy($id): void {
        $id = $this->requireId($id);
        if (!$this->model->find($id)) {
            $this->json(404, ['success' => false, 'message' => 'Data tidak ditemukan']);
        }

        $this->model->delete($id);
        $this->json(200, ['success' => true, 'message' => 'Data berhasil dihapus']);
    }

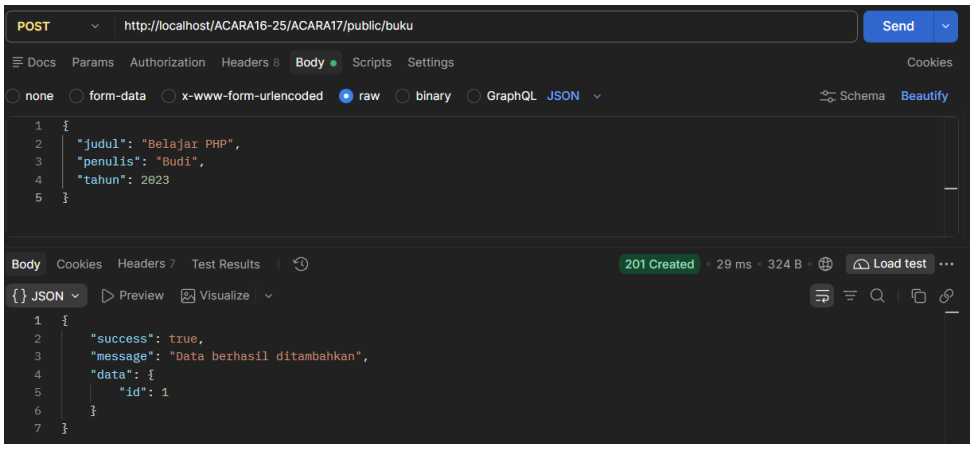
    /** Validasi judul, penulis (wajib) dan tahun (angka 4 digit, 1000 s.d. tahun depan). */
    private function validated(array $in): array {
        $judul   = isset($in['judul'])   && is_scalar($in['judul'])   ? trim((string) $in['judul'])   : '';
        $penulis = isset($in['penulis']) && is_scalar($in['penulis']) ? trim((string) $in['penulis']) : '';
        $tahun   = isset($in['tahun'])   && is_scalar($in['tahun'])   ? trim((string) $in['tahun'])   : '';

        if ($judul === '' || $penulis === '' || $tahun === '') {
            $this->json(400, ['success' => false, 'message' => 'Field judul, penulis, dan tahun wajib diisi']);
        }
        if (!ctype_digit($tahun) || (int) $tahun < 1000 || (int) $tahun > (int) date('Y') + 1) {
            $this->json(400, ['success' => false, 'message' => 'Tahun harus berupa angka 4 digit yang valid']);
        }
        if (mb_strlen($judul) > 150 || mb_strlen($penulis) > 100) {
            $this->json(400, ['success' => false, 'message' => 'Judul maksimal 150 karakter, penulis maksimal 100 karakter']);
        }

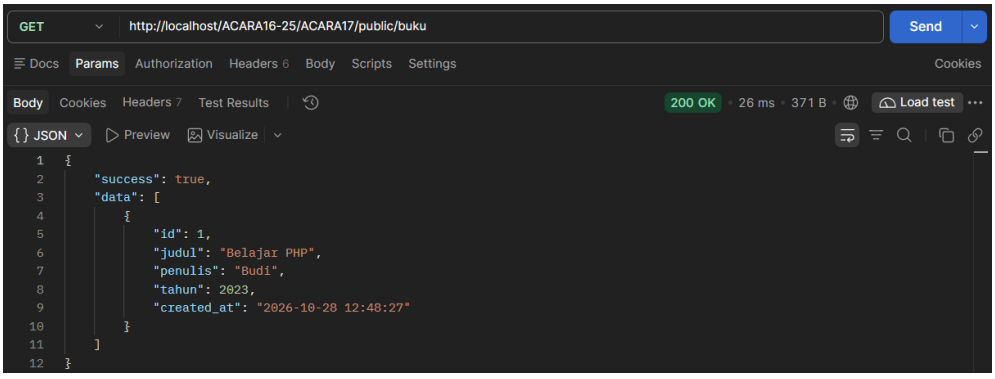
        return ['judul' => $judul, 'penulis' => $penulis, 'tahun' => (int) $tahun];
    }
}
```



[SCREENSHOT 24: kode BukuController.php atau Buku.php]



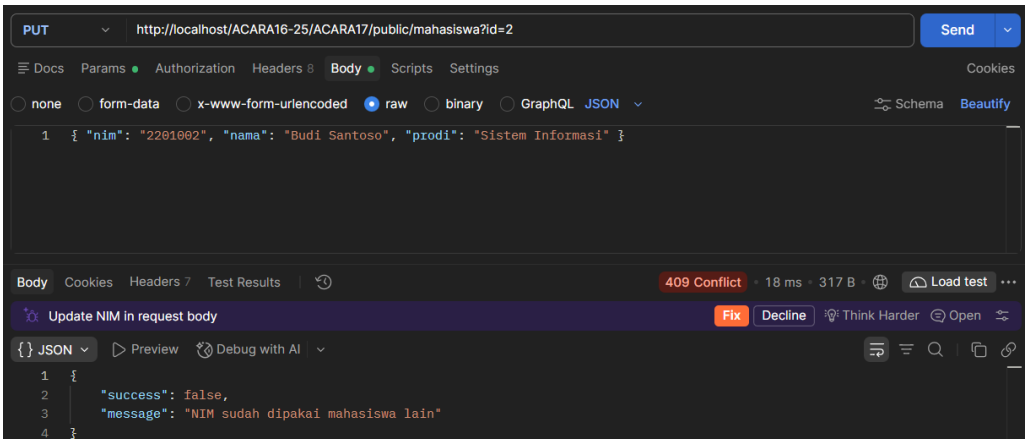
[SCREENSHOT 25: Postman POST /buku, status 201]



[SCREENSHOT 26: Postman GET /buku, status 200]

Tugas 2: Validasi NIM tidak boleh duplikat. Validasi dibuat berlapis. Pertama, Model memiliki method nimExists($nim, $exceptId) yang menghitung jumlah data dengan

NIM yang sama memakai prepared statement. Pada proses update, $exceptId membuat NIM milik data itu sendiri tidak dianggap duplikat. Kedua, Controller memanggil method tersebut sebelum menyimpan dan membalas 409 jika NIM sudah ada. Ketiga, tabel memiliki UNIQUE KEY sebagai pengaman terakhir. Pengujiannya sudah ditunjukkan pada bagian 4.8 (SCREENSHOT 20). Tambahkan juga satu pengujian PUT yang memakai NIM milik mahasiswa lain, hasilnya 409 “NIM sudah dipakai mahasiswa lain”.



[SCREENSHOT 27: Postman PUT dengan NIM milik mahasiswa lain, status 409]

Tugas 3: Dokumentasi endpoint dan diagram alur.
Tabel endpoint (resource mahasiswa; resource buku punya pola yang sama di /buku):

- GET /mahasiswa : tampilkan semua data, sukses 200
- GET /mahasiswa?id=1 : tampilkan satu data, sukses 200
- POST /mahasiswa : tambah data, sukses 201
- PUT /mahasiswa?id=1 : ubah data, sukses 200
- DELETE /mahasiswa?id=1 : hapus data, sukses 200

Contoh request POST:
{“nim”:”2201001",”nama”:”Andi Saputra”,”prodi”:”Teknik Informatika”}
Contoh response POST (201):
{“success”:true,”message”:”Data berhasil ditambahkan”,”data”:{“id”:1}}
Contoh response GET (200):
{“success”:true,”data”:[{“id”:1,”nim”:”2201001",”nama”:”Andi Saputra”,”prodi”:”Teknik Informatika”,”created_at”:”...”}]}
Contoh response error (404):
{“success”:false,”message”:”Data tidak ditemukan”}

Pembahasan

1. Front Controller membuat semua request melewati satu pintu, sehingga routing, penanganan error, dan pengaturan awal cukup ditulis sekali. URL juga menjadi bersih tanpa .php.
2. Pemisahan lapisan membuat tanggung jawab jelas: Router memilih Controller, Controller mengurus validasi dan response, Model mengurus SQL. Menambah resource Buku pada tugas mandiri hanya butuh satu Model, satu Controller, dan satu baris di peta Router. Hal ini membuktikan bahwa pola MVC mudah dikembangkan.
3. Prepared statement dipakai di semua query sehingga data pengguna tidak pernah disisipkan langsung ke teks SQL, dan SQL Injection tercegah.
4. Penempatan app/ di luar public/ melindungi kode sumber dan kredensial database.

5. Pada kode awal modul, PUT dan DELETE tanpa ?id atau dengan ID yang tidak ada tetap dianggap sukses. Pada implementasi ini ditambahkan pemeriksaan sehingga ID tidak valid dibalas 400 dan ID yang tidak ditemukan dibalas 404. Ini membuat hasil API lebih jujur bagi client.
6. Pembentukan response di Controller masih sederhana (ditulis langsung). Pada Acara 18 format ini akan dirapikan menjadi class Response tersendiri.
7. Postman memudahkan pengujian karena request, body JSON, dan hasil bisa disimpan dalam Collection dan dijalankan ulang tanpa membuat frontend.

E. KESIMPULAN

API CRUD menggunakan PHP Native berhasil dibangun dengan menerapkan pola arsitektur MVC (*Model-View-Controller*) dan Front Controller, dilengkapi dengan Autoloader dan Router untuk mengatur pemuatan class serta pengelolaan endpoint. Seluruh operasi CRUD pada resource mahasiswa, meliputi GET, POST, PUT, dan DELETE, berhasil diuji menggunakan Postman dengan response JSON dan HTTP status code yang sesuai, yaitu 200, 201, 400, 404, 405, 409, dan 500. Pemisahan tanggung jawab antara Model, Controller, dan Router juga telah diterapkan dengan baik, yaitu query SQL hanya berada di lapisan Model, sedangkan Controller bertugas menangani validasi dan response. Tugas mandiri berhasil diselesaikan dengan menambahkan resource Buku menggunakan pola yang sama, menerapkan pencegahan NIM duplikat yang menghasilkan status code 409, serta menyusun dokumentasi endpoint dan diagram alur API. Struktur yang telah dibangun ini menjadi fondasi yang baik untuk pengembangan pada materi berikutnya, seperti implementasi class Response, autentikasi, dan View, sehingga aplikasi dapat dikembangkan menjadi lebih terstruktur, konsisten, dan mudah dipelihara.